



Sprint 6: Deployment

API, AWS e Integración del MVP

Sprint Backlog, Roles y Definition of Done

Pedro Shiguihara, pshiguihara@pucp.edu.pe

15 de mayo de 2026

Agenda

1. Contexto: del reporte de negocio al MVP en producción
2. Metodología: CRISP-DM y Scrum aplicados al Sprint 6
3. Product Backlog: ítems del Sprint 6
4. Sprint 6: Alcance y Sprint Planning
5. Roles del equipo (5 personas)
6. Detalle de tareas por rol
7. Entregables del Sprint 6
8. Criterios de aceptación y Definition of Done

Contexto del Sprint 6

Del Sprint 5 (evaluar valor) al Sprint 6 (desplegar MVP)

🎯 Objetivo del Sprint 6

Empaquetar el modelo final como una **API REST**, desplegarla como un **MVP en AWS** (EC2 o Lambda), e integrarla con el **dashboard** construido en el Sprint 5 para que los stakeholders consuman predicciones en vivo. Cerrar el ciclo CRISP-DM con un sistema funcional y monitoreado.

➡ Insumos del Sprint 5

- `handoff/model/final_model.pkl`
- `handoff/model/preproc_pipeline.pkl`
- `handoff/contracts/input_schema.json`
- `handoff/contracts/output_schema.json`
- `handoff/contracts/example_request.json`
- `handoff/requirements.txt`
- `dashboard/app.py` (dashboard local)
- `reports/final_report.pdf` (recomendaciones de negocio)

🔧 Herramientas Clave

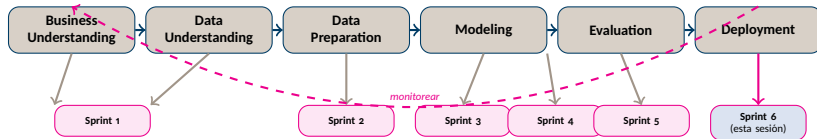
- 🐍 Python 3.10+
- 📋 FastAPI / Flask + uvicorn
- 🐳 Docker (imagen reproducible)
- 📦 AWS EC2, Lambda, ECR, S3
- 🛡️ AWS IAM (roles + secretos)
- 📊 CloudWatch (logs + métricas)
- 🔗 GitHub Actions (CI/CD)

⚠ Clave

Este sprint cierra el proyecto con un **MVP funcional** en AWS. Foco en **reproducibilidad** (Docker), **seguridad** (IAM, secretos) y **observabilidad** (logs, métricas, alertas).

Metodología: CRISP-DM dentro de Scrum

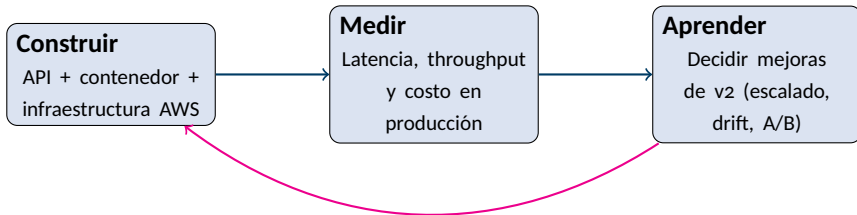
Mapeo de fases CRISP-DM a 6 Sprints — foco en Sprint 6



- **Sprint 1:** Business & Data Understanding.
- **Sprint 2:** Data Preparation.
- **Sprint 3:** Modeling baseline.
- **Sprint 4:** Modeling avanzado + tuning.
- **Sprint 5:** Evaluation + Business Value.
- **Sprint 6:** Deployment en AWS (*esta sesión*).

Ciclo Lean Startup aplicado al Sprint 6

Construir → Medir → Aprender



💡 Filosofía del Sprint 6

Un modelo que no llega a producción no genera valor. El Sprint 6 cierra el bucle CRISP-DM: empaqueta, despliega y conecta el modelo con el negocio. El énfasis no es en la sofisticación, sino en **ponerlo en marcha (MVP)** y dejar la **infraestructura lista** para iterar sobre datos reales.

Product Backlog Completo

Historias de usuario para todo el proyecto (6 sprints)

ID	Historia de Usuario	Fase CRISP-DM	Sprint
PB-01	Definir el problema de negocio y objetivos	Business Underst.	1
PB-02	Identificar criterios de éxito del modelo	Business Underst.	1
PB-03	Cargar y explorar el dataset inicial	Data Understanding	1
PB-04	Analizar calidad de datos (nulos, tipos, duplicados)	Data Understanding	1
PB-05	Generar estadísticas descriptivas y visualizaciones	Data Understanding	1
PB-06	Construir prototipo interactivo de exploración	Data Understanding	1
PB-07	Limpiar y transformar variables	Data Preparation	2
PB-08	Realizar feature engineering	Data Preparation	2
PB-09	Balancear clases si es necesario	Data Preparation	2
PB-10	Entrenar modelos baseline de clasificación	Modeling	3
PB-11	Evaluar modelos baseline con métricas	Modeling	3
PB-12	Comparar modelos y seleccionar candidatos	Modeling	3
PB-13	Optimizar hiperparámetros del mejor modelo	Modeling	4
PB-14	Aplicar técnicas avanzadas (ensambles, stacking)	Modeling	4
PB-15	Validar modelo final (cross-validation, test set)	Modeling	4
PB-16	Evaluar Business Value del modelo seleccionado	Evaluation	5
PB-17	Construir dashboard interactivo de resultados	Evaluation	5
PB-18	Documentar hallazgos y recomendaciones de negocio	Evaluation	5
PB-19	Empaquetar modelo como API (Flask/FastAPI)	Deployment	6
PB-20	Desplegar prototipo MVP en AWS (EC2/Lambda)	Deployment	6
PB-21	Integrar modelo con dashboard desplegado	Deployment	6

→ Las filas resaltadas en azul corresponden al **Sprint 6** (esta iteración): PB-19, PB-20 y PB-21.

Sprint 6: Sprint Planning

Alcance: Deployment — API, AWS e integración con el dashboard

Meta del Sprint

Empaquetar el modelo final como una **API REST**, contenerizarla con **Docker**, desplegarla en **AWS** (EC2 o Lambda) detrás de un endpoint público/privado, e integrarla con el **dashboard** del Sprint 5. Resultado: un MVP en producción con monitoreo básico.

Duración

- **1 semana** de trabajo colaborativo
- Daily stand-ups (async vía GitHub Issues)
- Sprint Review final con **demo en vivo** del MVP

Sprint Backlog (3 ítems grandes)

- PB-19** Empaquetar el modelo como API REST con FastAPI/Flask, contenerizar con Docker.
- PB-20** Desplegar el contenedor en AWS (EC2 o Lambda + API Gateway) con IAM y secretos.
- PB-21** Integrar el dashboard del Sprint 5 con la API desplegada y validar end-to-end.

Gestión

- Mismo repositorio GitHub de Sprints 1–5
- Cada sub-tarea = 1 Issue en el board Kanban
- Ramas por feature: `feat/PB-XX`

Roles del Equipo: 5 Personas

Roles adaptados al Sprint 6 (Deployment)



Nota importante

Los roles **rotan** nuevamente respecto al Sprint 5: cada miembro del equipo asume un rol distinto para ganar experiencia en la fase de Deployment. Todos colaboran y revisan el trabajo de los demás vía **Pull Requests**.

Rol 1: Project Manager

 Coordinación, planificación y seguimiento

Tareas del Sprint 6

1. Crear los Issues del Sprint 6 (PB-19, PB-20, PB-21) y sub-issues por tarea.
2. Reorganizar el board Kanban (cerrar Sprint 5, abrir Sprint 6).
3. Verificar que el paquete `handoff/` del Sprint 5 está completo (modelo + contratos + SLAs).
4. Coordinar la secuencia: API → AWS → integración.
5. Validar el **presupuesto AWS** disponible y los límites del Free Tier.
6. Programar la **demo en vivo** del MVP para el Sprint Review final.
7. Documentar el cierre del proyecto: lecciones aprendidas y backlog futuro (v2).

Estructura del Repo (Sprint 6)

```
proyecto/  
+-- api/  
|   +-- main.py  
|   +-- schemas.py  
|   +-- predict.py  
|   +-- Dockerfile  
|   +-- requirements.txt  
+-- infra/  
|   +-- terraform/ (o cloudformation)  
|   +-- deploy.sh  
|   +-- .github/workflows/cd.yml  
+-- dashboard/  
|   +-- app.py (consume API)  
+-- monitoring/  
|   +-- cloudwatch_alarms.json  
|   +-- dashboards.json  
+-- handoff/ (input del Sprint 5)  
+-- models/final_model.pkl
```

Rol 2: API Engineer

 Empaquetar el modelo como API (PB-19)

Tareas del Sprint 6

1. Crear el servicio **FastAPI** (`api/main.py`) con endpoints `/health`, `/predict` y `/version`.
2. Definir los **schemas** de entrada/salida con Pydantic según `handoff/contracts/`.
3. Cargar el modelo y el `preproc_pipeline.pkl` **una sola vez** al iniciar el proceso.
4. Implementar **validación** de inputs y manejo de errores con códigos HTTP claros.
5. Escribir el Dockerfile (imagen `python:3.10-slim`) y `requirements.txt` ajustado.
6. Probar localmente con `docker build + docker run` y `curl` contra los contratos.

`api/main.py` (FastAPI)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib, os

MODEL_PATH = os.getenv(
    'MODEL_PATH',
    'models/final_model.pkl'
)
model = joblib.load(MODEL_PATH)

app = FastAPI(title="Modelo Final v1")

class Features(BaseModel):
    age: int
    income: float
    region: str

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(x: Features):
    try:
        proba = model.predict_proba(
            [[x.age, x.income, x.region]]
            [0,1]
        )
        return {"proba": float(proba),
                "label": int(proba >= 0.35)}
    except Exception as e:
        raise HTTPException(400, str(e))
```

Rol 2: API Engineer — Dockerfile

 Imagen reproducible y pequeña

? Buenas prácticas

- Usar **python:3.10-slim** (pequeña, suficiente).
- Instalar primero `requirements.txt`, luego copiar código: aprovecha cache de Docker.
- Correr como **usuario no-root** (USER `appuser`).
- Exponer un único puerto (EXPOSE `8000`).
- Definir **HEALTHCHECK** para que ECS/ALB sepa si el contenedor está sano.
- No incluir `models/` en la imagen si pesa mucho: montar desde **S3** o usar volumen.

</> api/Dockerfile

```
FROM python:3.10-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY api/requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

COPY api/ ./api/
COPY models/final_model.pkl \
    /app/models/final_model.pkl

RUN useradd -m appuser
USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s \
    CMD curl -f http://localhost:8000/health \
    || exit 1

CMD ["uvicorn", "api.main:app", \
    "--host", "0.0.0.0", \
    "--port", "8000"]
```

Rol 3: Cloud / DevOps Engineer

 Despliegue del MVP en AWS (PB-20)

Tareas del Sprint 6

1. Decidir la arquitectura: **EC2** (más control) vs. **Lambda + API Gateway** (sin servidores). Para MVP: Lambda si la carga es baja y el modelo cabe; EC2 si el modelo es grande o hay dependencias pesadas.
2. Crear el **repositorio ECR** y subir la imagen Docker.
3. Configurar **IAM roles** mínimos: solo permisos requeridos.
4. Almacenar secretos (API keys) en **Secrets Manager** o **Parameter Store**.
5. Subir el `final_model.pkl` a un bucket **S3** versionado.
6. Crear la pipeline **CI/CD** con GitHub Actions: `push main` → `build` → `ECR` → `deploy`.
7. Configurar **HTTPS** (ACM + ALB o API Gateway) y un endpoint público estable.

Despliegue rápido (EC2 + Docker)

```
# infra/deploy.sh
set -e

REGION=us-east-1
ACCOUNT=123456789012
REPO=modelo-final

aws ecr get-login-password \
  --region $REGION | \
  docker login --username AWS \
  --password-stdin \
  $ACCOUNT.dkr.ecr.$REGION.amazonaws.com

docker build -t $REPO ./api
docker tag $REPO:latest \
  $ACCOUNT.dkr.ecr.$REGION.amazonaws.com/\
  $REPO:latest
docker push \
  $ACCOUNT.dkr.ecr.$REGION.amazonaws.com/\
  $REPO:latest

aws ssm send-command \
  --document-name "AWS-RunShellScript" \
  --targets "Key:tag:Name,Values=mvp-ec2" \
  --parameters 'commands=[
    "docker pull $REPO:latest",
    "docker run -d -p 80:8000 $REPO:latest"
  ]'
```

Rol 3: Cloud / DevOps — CI/CD con GitHub Actions

 Cada push `main` despliega automáticamente

¿Por qué CI/CD?

- Eliminar el despliegue manual: reduce errores humanos.
- Cada cambio en `main` produce una imagen nueva, etiquetada con SHA del commit.
- Habilita rollback inmediato: `aws ecs update-service --task-definition vN-1`.
- Sienta las bases para futuros **tests automáticos** y **aprobación manual** antes de prod.
- Las credenciales AWS se guardan como **secrets** de GitHub, no en código.

.github/workflows/cd.yml

```
name: CD - Deploy MVP
on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/
        configure-aws-credentials@v4
        with:
          aws-access-key-id:
            ${ secrets.AWS_KEY }
          aws-secret-access-key:
            ${ secrets.AWS_SECRET }
          aws-region: us-east-1
      - run: |
          cd api
          docker build -t modelo-final .
          docker tag modelo-final:latest \
            $ECR_URL:${ github.sha }
          docker push \
            $ECR_URL:${ github.sha }
      - run: bash infra/deploy.sh \
        ${ github.sha }
```

Rol 4: Integration Engineer

 Integrar el dashboard con la API desplegada (PB-21)

Tareas del Sprint 6

1. Modificar el `dashboard/app.py` para que **consume la API REST** en lugar del modelo local.
2. Configurar la URL del endpoint y la API key vía variables de entorno (no hardcodear).
3. Implementar **caché local** (`st.cache_data`) para evitar llamadas duplicadas.
4. Manejar **errores de red** (timeouts, 5xx) con mensajes amigables al usuario.
5. Desplegar el dashboard en **Streamlit Cloud, ECS Fargate** o un EC2 separado.
6. Validar el flujo end-to-end: usuario sube CSV → dashboard llama API → muestra predicciones + SHAP.

dashboard consume API

```
import os, requests
import streamlit as st
import pandas as pd

API_URL = os.getenv(
    "API_URL",
    "https://api.example.com"
)
API_KEY = os.getenv("API_KEY")

@st.cache_data(ttl=60)
def predecir(payload: dict) -> dict:
    r = requests.post(
        f"{API_URL}/predict",
        json=payload,
        headers={"x-api-key": API_KEY},
        timeout=5
    )
    r.raise_for_status()
    return r.json()

uploaded = st.file_uploader(
    "Sube un CSV", type='csv')
if uploaded:
    df = pd.read_csv(uploaded)
    preds = [predecir(r.to_dict())
              for _, r in df.iterrows()]
    df['pred'] = [p['label'] for p in preds]
    st.dataframe(df)
```

Rol 4: Integration Engineer — Pruebas end-to-end

 Validar el camino completo del usuario

¿Qué probar?

- **Smoke test:** GET /health responde 200 y latencia <1s.
- **Contrato:** POST /predict con example_request.json produce el formato esperado.
- **Carga básica:** 100 requests concurrentes (locust / ab) se manejan sin errores.
- **Timeout:** si el modelo tarda >2s, el dashboard muestra mensaje claro al usuario.
- **Auth:** peticiones sin API key son rechazadas (401).
- **Rollback:** el sistema vuelve a la versión anterior si el deploy falla el smoke test.

Test de carga con locust

```
from locust import HttpUser, task, between
import json

with open(
    'handoff/contracts/'
    'example_request.json') as f:
    payload = json.load(f)

class MVPUser(HttpUser):
    wait_time = between(0.5, 1.5)
    headers = {"x-api-key": "demo"}

    @task(3)
    def predict(self):
        self.client.post(
            "/predict",
            json=payload,
            headers=self.headers)

    @task(1)
    def health(self):
        self.client.get("/health")

# locust -f locustfile.py \
# --host https://api.example.com
```

Rol 5: Site Reliability & Monitoring

 Logs, métricas, alertas y observabilidad

Tareas del Sprint 6

1. Configurar **logs estructurados** (JSON) en la API y enviarlos a CloudWatch Logs.
2. Definir **métricas clave**: latencia (p50, p95, p99), throughput, tasa de error 5xx.
3. Crear un **dashboard CloudWatch** con esas métricas + uso de CPU/memoria del contenedor.
4. Configurar **alarmas**: error rate $> 2\%$ durante 5 min $\rightarrow$ SNS al equipo.
5. Establecer un **plan de monitoreo de drift**: log diario de la distribución de inputs vs. baseline del Sprint 2.
6. Documentar el **runbook**: cómo investigar incidentes, cómo hacer rollback, a quién escalar.

Logging estructurado

```
import logging, json, time
import logging.config

logging.basicConfig(
    level=logging.INFO,
    format='%(message)s')
log = logging.getLogger("api")

def log_event(event: str, **kw):
    log.info(json.dumps({
        "event": event,
        "ts": time.time(),
        **kw}))

@app.post("/predict")
def predict(x: Features):
    t0 = time.time()
    proba = model.predict_proba(
        [[x.age, x.income, x.region]])[0,1]
    log_event(
        "predict",
        latency_ms=(time.time()-t0)*1000,
        proba=float(proba))
    return {"proba": float(proba),
            "label": int(proba >= 0.35)}
```


Tablero Kanban en GitHub Projects

Visualización del flujo de trabajo del Sprint 6



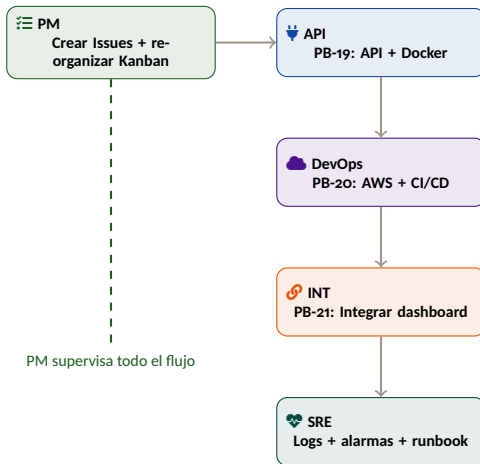
Flujo de Trabajo en el Kanban

Flujo

Cada Issue se mueve: **To Do** → se crea rama `feat/PB-XX` → **In Progress** → se abre Pull Request → **In Review** → se aprueba y mergea → **Done**. El SRE & Monitoring integra al final del sprint, una vez la API está desplegada y conectada al dashboard.

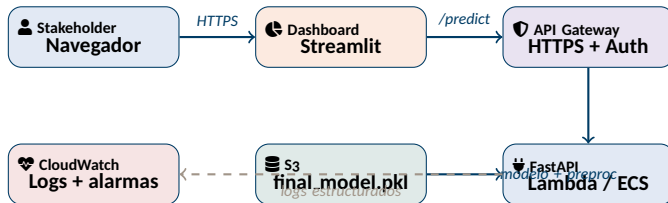
Dependencias entre Tareas

Orden sugerido de ejecución en el Sprint 6



Arquitectura del MVP en AWS

Visión integrada para Deployment



💡 Idea clave

Para un MVP, esta arquitectura cabe casi por completo en **AWS Free Tier**: Lambda + API Gateway + S3 + CloudWatch. EC2 con t2.micro también es elegible. La inversión principal es el **tiempo del equipo**, no el costo de infraestructura.

Tip 1: EC2 vs. Lambda — ¿cuál elegir?

 Compute para el MVP

? Comparativa

Aspecto	EC2 / Lambda
Modelo >250 MB	EC2 (Lambda no cabe)
Cold start	EC2 (sin) / Lambda (1s)
Costo a baja escala	Lambda (pago por uso)
Control total del SO	EC2
Auto-escalado	Lambda (nativo)
Tiempo de set-up	Lambda (más rápido)

</> Lambda con contenedor

```
# Dockerfile.lambda
FROM public.ecr.aws/lambda/python:3.10

COPY api/requirements.txt .
RUN pip install -r requirements.txt

COPY api/ ./api/
COPY models/final_model.pkl \
    ./models/final_model.pkl

CMD ["api.main.handler"]

# api/main.py (handler para Lambda)
from mangum import Mangum
from fastapi import FastAPI

app = FastAPI()
# ... endpoints igual que antes ...

handler = Mangum(app)
```

💡 Recomendación

20 Para un MVP con **tráfico bajo y modelo pequeño**, Lambda + API Gateway es la opción más simple y barata. Si el

Tip 2: Seguridad mínima del MVP



Lo que NO puede faltar antes de exponer la API

? Checklist de seguridad

- **HTTPS** en todos los endpoints (ACM + ALB / API Gateway).
- **API Key** o **IAM auth** (no endpoints abiertos).
- **IAM roles mínimos**: solo permisos requeridos por la API.
- **Secrets Manager** para credenciales: nunca en código ni en variables de entorno expuestas.
- **VPC** si el modelo accede a recursos privados; **security groups** restrictivos.
- **Rate limiting** (API Gateway) para evitar abuso o DoS accidental.
- **Logs auditables** con user-agent + IP de cada request.

</> Verificación de API key

```
import os
from fastapi import (
    Depends, Header, HTTPException)

EXPECTED_KEY = os.getenv("API_KEY")

def verify_key(
    x_api_key: str = Header(None)):
    if not EXPECTED_KEY:
        raise HTTPException(
            500, "API key no configurada")
    if x_api_key != EXPECTED_KEY:
        raise HTTPException(
            401, "API key invalida")

@app.post("/predict",
    dependencies=[
        Depends(verify_key)])
def predict(x: Features):
    ...
```

Tip 3: Versionado de la API y del modelo

 Que el cliente sepa qué modelo le respondió

¿Por qué versionar?

- Si reentrenas el modelo, el dashboard debe **saber** cuál está respondiendo.
- Permite **A/B testing** de modelos en producción (v1 vs. v2 con % de tráfico).
- Habilita **rollback**: redirigir el tráfico al :v1 si :v2 falla.
- Hace reproducible el debugging: *“este caso fue clasificado por la versión X del commit Y”*.

Práctica

Incluir en cada respuesta de /predict los campos model_version y api_version, y exponer un endpoint /version para diagnóstico.

Endpoint /version

```
import os, hashlib

API_VERSION = "1.0.0"
MODEL_VERSION = os.getenv(
    "MODEL_VERSION", "unknown")
MODEL_PATH = os.getenv(
    "MODEL_PATH",
    "models/final_model.pkl")

def model_hash():
    h = hashlib.sha256()
    with open(MODEL_PATH, "rb") as f:
        h.update(f.read())
    return h.hexdigest()[:12]

@app.get("/version")
def version():
    return {
        "api_version": API_VERSION,
        "model_version": MODEL_VERSION,
        "model_sha": model_hash(),
    }
```

Tip 4: Plan de monitoreo post-deploy

💖 Qué mirar la primera semana del MVP

? Métricas críticas

- **Latencia p95:** < SLA acordado (p. ej. 500 ms).
- **Error rate 5xx:** < 1% sostenido.
- **Throughput:** requests por minuto vs. capacidad.
- **Costo diario AWS:** alarma si supera el límite acordado.
- **Distribución de probas:** ¿cambia respecto al test set?
- **Tasa de aciertos** (cuando hay feedback): mantener un canal para etiquetas reales y comparar.

</> Alarma CloudWatch (CLI)

```
aws cloudwatch put-metric-alarm \  
  --alarm-name api-error-rate-high \  
  --metric-name 5XXError \  
  --namespace AWS/ApiGateway \  
  --statistic Sum \  
  --period 300 \  
  --threshold 5 \  
  --comparison-operator \  
    GreaterThanThreshold \  
  --evaluation-periods 1 \  
  --alarm-actions \  
    arn:aws:sns:us-east-1:123:OnCall \  
  --dimensions \  
    Name=ApiName,Value=modelo-final
```


Errores comunes en Deployment

 Qué evitar y cómo resolverlo

	Error común	Solución
⊘	Cargar el modelo en cada request	Cargarlo una sola vez al iniciar el proceso (alcance global)
⊘	Hardcodear API keys o ARNs en el código	Usar Secrets Manager / variables de entorno + GitHub secrets
⊘	Imagen Docker enorme con dependencias innecesarias	Usar <code>python:3.10-slim</code> y <code>multi-stage build</code> ; podar <code>requirements.txt</code>
⊘	Desplegar sin endpoint <code>/health</code>	Sin health check, ALB/ECS no detectan contenedores rotos
⊘	No versionar el modelo en la API	Imposible debuguear: la respuesta no dice qué modelo respondió
⊘	Lanzar a producción sin alarmas ni runbook	Definir alarmas de error/latencia y runbook antes del go-live

Entregables del Sprint 6

Lo que cada rol debe entregar al final de la semana

Rol	Entregable	Formato
PM	Issues del Sprint 6 cerrados, board final, retrospectiva del proyecto	GitHub Project + retro.md
API	Servicio FastAPI con /health, /predict, /version y Dockerfile probado localmente	api/ + imagen Docker
DevOps	API desplegada en AWS con endpoint público HTTPS y pipeline CI/CD	Endpoint URL + .github/workflows/cd.yml
INT	Dashboard consumiendo la API en producción, validado end-to-end	dashboard/ desplegado
SRE	Logs estructurados, dashboard CloudWatch, alarmas y runbook de incidentes	monitoring/ + runbook.md

Definition of Done (DoD)

Criterios que cada tarea debe cumplir para considerarse terminada

✓ Criterios Generales

- ☐ Código en rama `feat/PB-XX` con Pull Request aprobado y mergeado a `main`.
- ☐ Pipeline CI/CD verde: build + push de imagen + deploy automático.
- ☐ Documentación actualizada (`README`, `runbook.md`).
- ☐ Issue correspondiente movido a "Done" en el board.
- ☐ Variables sensibles fuera del repo (Secrets Manager / GitHub secrets).

★ Criterios específicos del Sprint 6

- ☐ La API responde 200 en `/health` y produce predicciones consistentes con `example_request.json`.
- ☐ Imagen Docker construida, etiquetada con SHA del commit y subida a ECR.
- ☐ Endpoint público HTTPS protegido por API Key o IAM.
- ☐ Dashboard desplegado consume la API real, no el modelo local.
- ☐ Dashboard CloudWatch con latencia p95, error rate y costo diario; al menos 1 alarma activa.
- ☐ Demo en vivo del MVP completada en el Sprint Review final.

Visión General: 6 Sprints hasta el Despliegue

Del entendimiento del negocio al MVP desplegado en AWS

Sprint	Fase CRISP-DM	Objetivo principal
1	Business + Data Underst.	Comprender el problema y explorar los datos
2	Data Preparation	Limpiar, transformar y crear features
3	Modeling	Entrenar modelos baseline y comparar
4	Modeling	Optimizar hiperparámetros y modelos avanzados
5	Evaluation	Evaluar Business Value y documentar resultados
6	Deployment	Desplegar el mejor modelo como MVP en AWS

Meta del Sprint 6

Cerrar el ciclo CRISP-DM con un **MVP desplegado, monitoreado y conectado al dashboard**, listo para ser usado por stakeholders reales y para iterar en una versión 2 sobre datos de producción.

Resumen del Sprint 6

❓ ¿Qué?

Deployment: API + AWS +
integración con dashboard

👥 ¿Quién?

5 roles: PM, API,
DevOps, INT, SRE

⚙️ ¿Cómo?

FastAPI + Docker +
AWS + CloudWatch



Entregable

API + endpoint HTTPS
+ dashboard integra-
do + monitoreo activo



Siguiente

V2: A/B testing, drift en vi-
vo, escalado automático

! Recordar

- El Sprint 6 **no entrena modelos**: usa el modelo final del Sprint 4 y los contratos del Sprint 5.
- El énfasis es **reproducibilidad** (Docker), **seguridad** (IAM, secretos) y **observabilidad** (logs, alarmas).
- Un MVP simple en producción vence a una arquitectura sofisticada sin desplegar.
- Documentar el **runbook** para que cualquier miembro del equipo pueda operar el sistema.

Checklist de Inicio

Pasos inmediatos para comenzar el Sprint 6

1. **PM:** Cerrar el Sprint 5 en el board y crear los Issues del Sprint 6 (PB-19, PB-20, PB-21 y sub-issues).
2. **PM:** Reasignar roles para que cada miembro experimente la fase de Deployment.
3. **Todos:** `git pull && dvc pull` para sincronizar el estado del Sprint 5.
4. **Todos:** Verificar que el paquete `handoff/` contiene modelo, contratos y SLAs (sin esto, el sprint se bloquea).
5. **API:** Iniciar `api/main.py` y `api/Dockerfile` en rama `feat/PB-19`; smoke test con `docker run + curl`.
6. **DevOps:** Crear el repo ECR y la pipeline `.github/workflows/cd.yml` en rama `feat/PB-20`; primer despliegue manual end-to-end.
7. **INT:** Cuando la API tenga endpoint público, modificar `dashboard/app.py` (rama `feat/PB-21`) y validar end-to-end.

Checklist de Inicio (cont.)

Pasos inmediatos para comenzar el Sprint 6

8. **SRE:** Configurar logs estructurados, dashboard CloudWatch, al menos una alarma activa y redactar el `runbook.md` (rama `feat/SRE`).
9. **Todos:** Sprint Review final — demo en vivo del MVP, retrospectiva del proyecto y backlog de la versión 2 (escalado, drift en línea, A/B testing).



Diplomado de Especialización en **Data Analytics**